CCIT Report #458 December 2003

Iterative decoding of low-density parity check codes over compound channels

Ilan Sutskover and Shlomo Shamai (Shitz)

Abstract

We present a setting for decoding of LDPC codes jointly with channel estimation, suitable for transmission over memoryless compound channels. We show that the performance of the combined scheme can be rigorously evaluated by means of density evolution, and focus on density evolution as a tool for designing a channel estimator that matches not only to the channel, but also to the LDPC ensemble as well.

I. Introduction

Low-density parity-check (LDPC) codes have proven themselves extremely useful for error protection in a memoryless noisy channel when the channel statistics are known [1]. Iterative decoding, either of LDPC codes or of Turbo codes, has been considered also in conjunction with certain unknown channel parameters, e.g., [2], [3], [4] that address iterative decoding and channel estimation for Turbo codes and Rayleigh fading channels, [5] that deals with blind turbo equalization, [6] that examines Turbo codes over an AWGN channel with unknown phase rotation and [7], [8], [9], [10] that investigate LDPC decoding at the same channel conditions.

In this paper, decoding of LDPC codes over compound channels [11] is considered. Examples for such channels are: complex AWGN with phase uncertainty or frequency uncertainty (or both), block fading AWGN channel, BSC with unknown transition probability etc.

In the literature, several strategies are taken when parameters uncertainty exists. Among these strategies we can find the following non-exhaustive list: Schemes that improve their estimation from one coded-block to the next [6], [12], systems that patch SISO channel estimators to SISO decoders [6], [13], [14] and frameworks that use factor graphs to represent the channel parameter [8], [9], [10]. Numerical analysis of complete systems (joint decoding and estimation/equalization) appears to the best of our knowledge only in [8], [15], [16]. All other methods are analyzed via Monte-Carlo simulations.

The work most close in nature to the one presented in this paper is perhaps the work of Nuriyev and Anastasopoulos [8]. The authors of [8] present results for joint decoding and phase estimation when LDPC codes are used. Their setting, unlike the setting used in this paper, is Bayesian. In an AWGN, a constant phase, uniformly distributed over $[0, 2\pi]$, is assumed to govern short sub-blocks of the coded block (where BPSK transmission is regarded). Several message-passing decoders are proposed and analyzed via density evolution. However, where [8] puts its emphasis on the gain from pilot bits and its results are averaged with respect to a uniform distribution over the phase rotation, our work addresses a single unknown parameter (which can be, for example, an unknown phase rotation) and puts its emphasis on proper channel estimators design and the theoretical support for the analysis of the complete system by density evolution (which is somewhat more complicated than in the setting of [8]). Moreover, the setting of [8] is specialized to

This paper was presented in part at the 22nd Convention of Electrical Engineers in Israel, Tel Aviv, Israel, Dec. 1st 2002, and in part at the 3rd International Symposium on Turbo Codes & Related Topics, Brest, France, Sep. $1-5$, 2003.

block-noncoherent AWGN channels, while our setting is general and may apply to any set of memoryless compound channels[1].

The structure of this paper is as follows: A short survey of LDPC codes and iterative decoding is given in Section II. Then, the problem is stated in Section III and a unified solution approach is presented at Section IV. The presented approach is a message-passing algorithm that is independent of the specific channel estimator in use. Section V, which is one of the main contributions of this paper, introduces a concept that allows for channel estimator design process such that the estimator is not only channel dependent but also code dependent. Some specific examples are then given in Section VI. Section VII concludes.

II. Preliminaries

A bipartite graph is associated with each LDPC code (a detailed presentation can be found in [1]). *Variable nodes* are associated with the codeword's symbols. *Check nodes* are associated with the code's parity-check equations. Let λ be a *degree sequence*, i.e., $\lambda = \{\lambda_i\}_{i\geq 1}$ is a set of positive real numbers such that $\sum_i \lambda_i = 1$. Let ρ be another degree sequence. A (λ, ρ)-ensemble of LDPC graphs (each graph is associated with a unique LDPC code) is an ensemble of bipartite graphs such that the fraction of edges that incident a variable (resp. check) node of degree i is λ_i (resp. ρ_i). We restrict ourselves in this paper to binary LDPC codes only. Extension of the concept to symbols of larger alphabet are natural. Nevertheless, modulation by grouping several bits per symbol will be applied when appropriate.

The Sum-Product algorithm [17], [18] is typically used for decoding LDPC codes. It operates on the associated LDPC graph and it is a message passing algorithm; that is, messages are passed along edges of the graph, where a message emanating from a node is a function of all messages received at this node along all its connected edges except for the edge over which the output message is passed. The following is a short description of the algorithm:

- **Initialization:** Given a measurement vector, $Y_1^n \triangleq Y_1, \ldots, Y_n$, set the measurement-to-variable messages as

$$\forall c_j \in R(v_i) : m_{y_i v_i}(Y_i) = m_{v_i c_j}^{(0)} = \log \frac{p(Y_i|X_i = 1)}{p(Y_i|X_i = -1)}, \tag{1}$$

  where $R(v_i)$ is the set of check nodes connected to variable node v_i. These messages also serve as the initial messages. $p(\cdot)$ refers here to pdfs of continuous variables or pmfs of discrete variables.

- **Check-to-Variable messages:** For each check node c_j let $m_{v_i c_j}$ be its input message from variable node v_i. The output message $m_{c_j v_i}$ from the check node c_j to the variable node v_i at the ℓth iteration is given by

$$m_{c_j v_i}^{(\ell)} = 2 \tanh^{-1} \left(\prod_{v' \in L(c_j) - \{v_i\}} \tanh \left(\frac{m_{v' c_j}^{(\ell-1)}}{2} \right) \right) \tag{2}$$

  where $L(c_j)$ denotes the set of variable nodes connected to check node c_j.

[1]In fact, a finite set of representatives is used for the analysis and the design of the estimators. When sampled dense enough, it is expected that the performance of the system is meaningful even for compound channels with continuous alphabet parameters, such as the AWGN with constant unknown phase rotation.

- **Variable-to-Check messages:** For each variable node v_i the output message $m_{v_i c_j}$ to check node c_j at the ℓth iteration is given by

$$m_{v_i c_j}^{(\ell)} = m_{y_i v_i} + \sum_{c' \in R(v_i) - \{c_j\}} m_{c' v_i}^{(\ell)} \tag{3}$$

- **Termination:** For each variable node v_i the output message m_{v_i} is given by

$$m_{v_i}^{(\ell)} = m_{y_i v_i} + \sum_{c' \in R(v_i)} m_{c' v_i}^{(\ell)} \tag{4}$$

The algorithm starts with an initialization, and then iterates back and forth between check nodes and variable nodes. Parallel schedule is assumed throughout this paper. When the graph on which the Sum-Product algorithm operates is a tree, the output messages, $\{m_{v_i}\}$, are in one-to-one relation with the posterior probability of a bit given all available measurements, hence the algorithm is optimal with respect to bit error probability. Over any sub-graph which is a tree the passed messages represent true log-likelihood ratio (LLR) values at the first iterations (until the first instant a message traverse a cycle). Hard decoding values can therefore be obtained by slicing at zero.

When a LDPC code is uniformly drawn from a given (λ, ρ) ensemble, two important properties can be shown to hold [1]: a tree property and a concentration property. The tree property asserts that a randomly chosen edge in the graph spans a tree up to any fixed depth, with probability that approaches one as the block length approaches infinity. The concentration property asserts that after any finite ℓ iterations, the performance of any message passing decoder over a randomly selected graph approaches the average performance over the ensemble in probability, and exponentially fast. The concentration property implies that for large block lengths, the ensemble average performance is a good indicator for the performance of any individual code in the ensemble.

The tree property in conjunction with a memoryless channel imply independence among input messages toward any node, asymptotically. With simple operations such as Equations (2)-(3), the (conditional) density of the output of any node is easily obtained as a function of the (conditional) densities at the input. This technique, called *density evolution* [1] allows analysis of performance to any finite iteration with asymptotically large block length. In fact, in [1] it is shown that the performance of the Sum-Product algorithm (as well as many other admissible algorithms) is independent of the transmitted codeword; this simplifies the density evolution process, as only a single conditional density (conditioned on transmission of the zero codeword) needs to be tracked.

III. Problem formulation

In this paper we are interested in the performance of iterative decoding of LDPC codes over memoryless compound channels [11]. A memoryless compound channel is a family of channels, Θ, such that

$$\forall \theta \in \Theta : \; p(y_1^n | x_1^n; \theta) = \prod_{i=1}^{n} p(y_i | x_i; \theta),$$

where x_1^n is the channel input and y_1^n is the channel output. Here, the decoder is ignorant of the specific channel over which transmission is carried out, but is fully informed with the class of channels, Θ, to whom the channel belongs and of the code on which it operates. The encoder knows the specific channel θ and ignores the fact that the decoder is informed only with the set Θ. Such a case may arise, for example, when

the receiver itself introduces some uncertainty. Examples of compound channels are: BSC with unknown transition probability, AWGN channel with unknown noise variance, complex AWGN with unknown phase rotation etc. This paper does not address differential modulation techniques such as in [19]. Moreover, combination of blind iterative decoding and differential modulation in the presence of (perhaps partially) unknown channel should be carefully treated, as explained in [5, Sec. VII], but c.f. [20].

A good decoding technique that is ignorant of the realization θ is now sought.

IV. Unified structure of the proposed algorithms

The concept to be presented in this paper is the following. A new definition of ensembles is presented, which incorporates "channel estimation" ensemble to LDPC ensemble. The new ensemble contains bipartite graphs with three types of nodes, arranged in three layers: check nodes, variable nodes and estimator nodes. A message passing algorithm is performing the decoding + channel estimation, and a concentration property asserts that the decoding performance of an individual in the ensemble is approaching the ensemble average in probability and exponentially fast. The ensemble average performance of the decoder is then rigorously evaluated using density evolution.

Definition 1: An ensemble of joint code-estimator (bipartite) graphs is defined by the quartet $(\lambda, \rho, \eta, \mathsf{e})$, where λ,ρ and η are degree sequences and e is the number of estimator nodes. The degree sequences λ and ρ relate to the connectivity between variable nodes and check nodes. The degree sequence η holds the fractions of edges between variable nodes and estimator nodes with respect to the estimator node degrees (i.e., a "left" degree sequence). Each variable node is connected to a single estimator node via a single edge (see also Fig. 2).

Three-layered graphs are previously used in the literature (e.g., see [21], [22]). However, as far as we are aware, these graphs can be classified into two types: Graphs in which the third layer represents the channel effects in a format of a "code" (e.g., [21], [22]) and graphs in which the third layer consist of one node which is used for operations such as estimation or demodulation (e.g., [10]). The work of [8] utilizes many estimation nodes at the third layer, but only a single node for any single constant unknown parameter.

The fact that a variable node is connected to a single estimator node incurs no loss of generality; variable nodes connected to several estimator nodes are allowed to merge and the functionality of the merged node is at least as good as the functionality of the separation. Note also that if the degree sequence η is independent of the block length, the number of estimator nodes grows linearly with the block length as does the number of check nodes. However, since with respect to estimator nodes there is no "rate" constraint, Definition 1 may allow dependence of the sequence η on the block length.

The estimation subgraph is uniformly drawn according to η, independently of the drawn code subgraph, the latter is commonly uniformly drawn according to λ and ρ.

Interestingly, the system proposed above can be presented as an iteration-varying channel estimator operating jointly with a LDPC simple decoder, as depicted by Fig. 1. The main differences from the commonly known turbo iterations between a decoder and a channel estimator lie at the channel estimator "black-box". As expected, the estimator has as many outputs as inputs. However, the message-passing constraint renders each output to be a function of a few inputs, rather that a function of all inputs. Moreover, the transfer function employed by the estimator depends on the number of iteration and varies accordingly. Additionally, an interleaver may be placed between the LDPC decoder and the estimator. This interleaver is optional, and may be removed due to different considerations. However, it facilitates the density evolution analysis process, which is not bothered with the specific connectivity of the system.

The general structure of the suggested algorithm is the following:

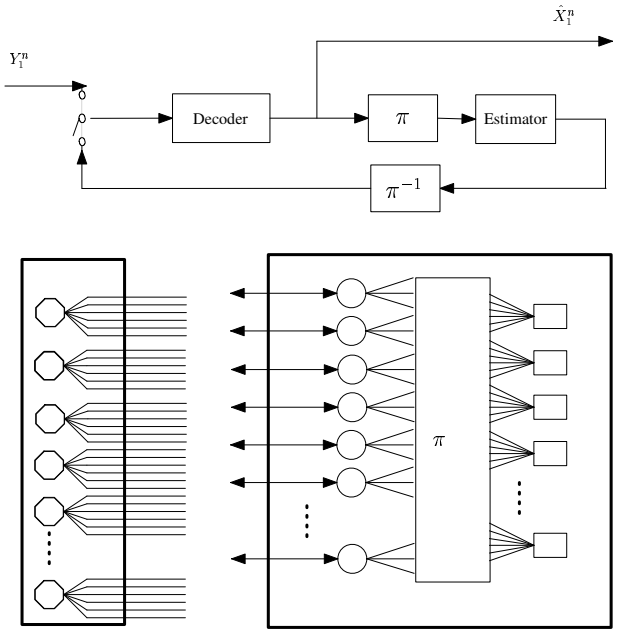


Fig. 1. The top figure shows the proposed system in a Turbo structure. The presented below are the "black boxes" of the decoder (right) and the estimator (left). An interleaver (denoted by π) may or may not reside between the decoder and the estimator.

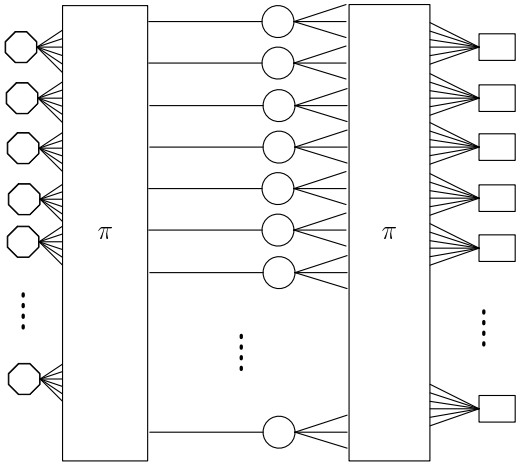


Fig. 2. The three layered graph associated with the proposed approach. The interleaving between variable nodes and estimator nodes may be excluded (see also Appendix I).

Algorithm 1: The algorithm operates on the graph depicted in Fig. 2. It starts with an initialization, and then iterates using a parallel scheduling: left-left-right-right.

- **Initialization:** Given a measurement vector, $Y_1^n \triangleq Y_1, \ldots, Y_n$, set the measurement-to-variable messages as

$$\forall c_j \in R(v_i) : m_{y_i v_i}(Y_i) = m_{v_i c_j}^{(0)} = f_1(Y_i) \tag{5}$$

for some function $f_1(\cdot)$.

- **Check-to-Variable messages:**

$$m_{c_j v_i}^{(\ell)} = 2\tanh^{-1}\left(\prod_{v'\in L(c_j)-\{v_i\}} \tanh\left(\frac{m_{v'c_j}^{(\ell-1)}}{2}\right)\right)$$

- **Variable-to-estimator messages:** For each variable node v_i the output message $m_{v_i e_k}$ to an estimator node e_k is given by

$$m_{v_i e_k}^{(\ell)} = \sum_{c'\in R(v_i)} m_{c'v_i}^{(\ell)}$$

- **Estimator-to-variable messages:** For each estimator node, e_k, the output message, $m_{e_k v_i}$, to a variable node v_i is given by

$$m_{e_k v_i}^{(\ell)} = f_2^{(\ell)}\left(\{m_{v'e_k}^{(\ell)}, Y_{v'}\}_{v'\in R(w_k)-\{v_i\}}, Y_{v_i}\right) \tag{6}$$

for some function $f_2^{(\ell)}(\cdot)$. We denote by $Y_{v'}$ the channel measurement that corresponds to variable node v'.

- **Variable-to-Check messages:**

$$m_{v_i c_j}^{(\ell)} = m_{e_{L(v_i)}v_i}^{(\ell)} + \sum_{c'\in R(v_i)-\{c_j\}} m_{c'v_i}^{(\ell)}$$

where $e_{L(v_i)}$ denotes the unique estimator node e_k that is connected to variable node, v_i.

- **Termination:** For each variable node v_i the output message m_{v_i} is given by

$$m_{v_i}^{(\ell)} = m_{e_{L(v_i)}v_i}^{(\ell)} + \sum_{c'\in R(v_i)} m_{c'v_i}^{(\ell)}$$

In general, the function $f_1(\cdot)$ needs to be chosen according to the type of the compound channel. It is reasonable, though, to choose $f_1(Y)$ to have the structure of the LLR function for a known realization, θ. In our simulations, the realization θ that forms the threshold for the known channel was used. For example, the LLR transform associated with a BIAWGN channel is of the format $\frac{2}{\sigma^2}Y$. For a $(3,6)$ LDPC code, the threshold channel is at $\sigma = 0.88$ [1], so we may choose to use $f_1(Y) = \frac{2}{0.88^2}Y$ for the case of BIAWGN channel with zero-mean and unknown noise variance. Similarly, for a BSC and a $(3,6)$ LDPC code (whose threshold is $\epsilon = 0.084$ [1]) we may choose for the compound BSC $f_1(Y) = Y\ln\left(\frac{1-0.084}{0.084}\right)$, where $Y\in\{\pm 1\}$. The function $f_2(\cdot)$ shall be described later, but it is important to notice that $f_2(\cdot)$ retains the desired message-passing nature of the algorithm.

Lemma 1 (Tree property): Consider a $(\lambda,\rho,\eta,\mathsf{e})$ ensemble, where the maximal estimator node degree grows at most sub-polynomially with the block length. Then, a randomly chosen edge in the graph spans a tree up to any fixed depth, with probability that approaches one as the block length approaches infinity.

With this property, which is proven at Appendix I we can claim independence of incoming messages when the compound channel is memoryless. Such independence is a sufficient prerequisite for rigorous analysis via density evolution. For any channel realization, $\theta\in\Theta$, the performance of the decoding process (which is done together with estimation steps) can be numerically evaluated. With the graph structure introduced above, this process can be carried out for any function employed by the estimation nodes. Note that in the graph associated with the constant phase model in [10] all variable nodes of a LDPC graph are attached to a single channel estimator node. There, a tree property does not exist, of course, but for a block length

approaching infinity, the effect of a single message on the total vanishes. Thus, asymptotically, the approach taken in [10] may yield rigorous analysis; however, for any finite block length and any finite number of iterations, the analysis of [10] is an approximation, whereas the analysis presented in this paper is exact until first cycles are closed.

The following concentration property shows that for the ensemble defined in Definition1, any message-passing algorithm whose variable-to-check messages relate to soft decisions of the symbols associated with the variable nodes has approximately the same performance. The property is proved at Appendix II. The proof technique, which was suggested in [1] (as an extension of [23]), does not extend to the single-node graph constellation of [10].

Proposition 1 (Concentration property): The decoding performance of an individual in the ensemble, using a message passing decoder, is approaching the ensemble average in probability and exponentially fast.

Assume all channels, $\theta \in \Theta$, are partially ordered by some order. For example, in [1] a relation of stochastic degradation [24, Eq. 14.201] was used as this order. Within each chain (which is well ordered) we define the chain threshold with respect to the decoder in use as the "best" channel that does not result in convergence to zero error probability. If the set Θ is well ordered by itself, the threshold of Θ is the (only-)chain threshold. The following compound channel examples are well ordered: BSC with unknown transition probability, AWGN channel with unknown noise variance, complex AWGN with unknown phase rotation.

With the definition above, the threshold always exists, but it is not clear how it is to be found. For the known channel case, monotonicity property was proven to hold for the Sum-Product decoder (relying on its optimality) and thus the threshold can be found up to any desired precision. In our case, monotonicity cannot rely on optimal decoding. Nevertheless, for any sampled grid of channels on a chain, $\{\theta_1, \ldots, \theta_n : \theta_i \in \Theta, 1 \leq i \leq n\}$, monotonicity can be tested by density evolution. Indeed, over the compound BSC and the complex AWGN with unknown phase rotation, monotonicity was verified for all our selected grids so far for channels "under the threshold" (i.e., channels for which successful decoding is possible.)

V. Design of channel estimator transfer functions

The discussion above did not address the transfer function of the estimator nodes. Indeed, as long as estimator nodes are message-passing, performance can be tracked for any transfer function. We now ask, which transfer function, $f_2^{(\ell)}(\cdot)$, is proper to be used?

Before getting into details, we emphasize that the methods considered in this paper do not employ, essentially, pilot symbols or training sequences (e.g., [2], [7], [25]). With respect to pilot symbols, two types of gains are conventionally sought for: improved channel estimation (see references before) and triggering convergence of iterative decoding [26] (even where the channel is fully known!); the term "code doping" is usually used for the latter task. It is interesting to observe that "hard" doping, where some bits that are fully known to the decoder are implanted into the codeword, is a redundant operation when LDPC codes are addressed (as opposed to Turbo codes for which doping was suggested at [26]). To see this, think of a regular LDPC code where some of its bits are known to the decoder. At the decoding process, the variable nodes associated with the known values produce constant messages (over a known channel). Thus, the decoding performance is identical to the case where these nodes are removed from the graph along with their adjacent edges. With this process, an irregular graph is obtained from the regular graph. The same argument holds when an initial irregular graph is considered, of course. However, for LDPC codes, design methods such as in [27] are rigorous and should provide much better results than the heuristic doping

process. Thus, for LDPC codes, pilot symbols serve only for channel estimation improvement. Their effect on performance is not within the scope of this paper, though this paper provides a comfortable neighborhood for the exploration of pilot symbols effect.

In general, several types of approaches may be suggested.

- A Bayesian approach may be taken if a probability law rules the selection of the channel realization, $\theta \in \Theta$. If the set Θ is finite, an asymptotically optimal decoder may be obtained by alphabet extension, solving for $P(X, \theta|Y)$ which allows evaluating $P(X|Y, \theta)$. However, this decoder requires passing messages of very high dimensionality. A normal LDPC graph, whose variable nodes correspond to the extended alphabet of (X, θ) is suitable for optimal decoding with the Sum-Product algorithm. The solution, however, yields an optimization that does not force the parameter, θ, to be the same for all symbols of the code, just as optimal per-symbol MAP algorithms does not necessarily outputs a legal codeword. Decoders that insist on constraining the parameter to be the same constant for all symbols typically operate on graph constructions such as in [10]. Here, analysis becomes rigor only for infinite block length.
- An arbitrary model may be chosen for the estimator node's output message, m_{ev}. Then, the estimator node is required only to calculate the model's parameters and use this model to update the output message. For example, a mixture of Gaussians model is suggested in [10].
- An empirical off-line relation can be used between the estimator node input messages and the identity of the realized channel. For example, in [6] a relation between a phase offset and the mean absolute value of the input LLR messages is found and used.
- Approximations to the known channel LLR values may be used. For example, see [28].
- Iteration-invariant estimators may be used. Their selection is typically only channel dependent. Following are three examples suitable for a compound BSC.

  Example 1: Let $\hat{m}_{v'e} = m_{v'e}\text{sign}(Y_{v'})$. Then the estimator function

$$m_{ev} = \text{sign}(Y_v) \cdot \log \frac{\sum_{v' \in R(e)-\{v\}} 1(\hat{m}_{v'} > 0) + 1/2}{\sum_{v' \in R(e)-\{v\}} 1(\hat{m}_{v'} < 0) + 1/2} \tag{7}$$

  is a hard-decision based version of the probability estimator according to Krichevski and Trofimov [29]. The multiplication by $\text{sign}(Y_v)$ relates the result to a LLR type output message.

  Example 2: One may generalize the Krichevski and Trofimov (KT) estimator to depend on soft values rather than hard decisions. The estimator

$$m_{ev} = \text{sign}(Y_v) \cdot \log \frac{\sum_{v' \in R(e)-\{v\}} (1 + e^{-\hat{m}_{v'}})^{-1}}{\sum_{v' \in R(e)-\{v\}} (1 + e^{\hat{m}_{v'}})^{-1}}$$

  can be shown to minimize $\sum_{j \neq i} D(p_j \parallel P(\tau))$ where $P(\tau)$ is a Bernoulli source with parameter τ that has to be fitted to a sequence of Bernoulli sources with parameters p_j.

  Example 3: The estimator

$$m_{ev} = \text{sign}(Y_v) \cdot \frac{1}{|R(e)| - 1} \sum_{v' \in R(e)-\{v\}} \hat{m}_{v'}$$

  can be shown to minimize $\sum_{j \neq i} D(P(\tau) \parallel p_j)$, where a single true source $P(\tau)$ is estimated by fitting it to other Bernoulli sources with parameters p_j.

Other examples include the MMSE-like estimation proposed by [2] for flat-fading channels and the Gibbs sampler employed by [5]. The latter is an iteration-invariant function of the inputs and the channel measurements, which is used in this case for blind turbo equalization and decoding of Turbo codes.

In addition to the proposed estimation methods described above, a new type of estimator is now suggested. This method utilizes density evolution not only for performance analysis but also for design of good channel estimators that are matched to both the channel and the given LDPC ensemble.

Suppose that $f_1(\cdot)$ is fixed and $\{f_2^{(l)}(\cdot)\}_{l<\ell}$ are given. Also, assume that some symmetry conditions are retained by all steps of the algorithm until iteration $\ell-1$, so that the performance of the algorithm is independent of the transmitted codeword and the all-zero codeword may be assumed to be transmitted; this latter assumption will be abandoned later. Let $\Upsilon \subseteq \Theta$ be a finite set of representatives from Θ.

Under the assumptions, for each $\theta \in \Upsilon$, densities (conditioned on transmission of the all-zero codeword) may be tracked by means of density evolution. Then, some statistics of the input messages is defined (e.g., $\bar{m}_v^{(\ell)} = \frac{1}{|R(e)|-1}\sum_{v' \in R(e)-\{v\}} m_{v'e}^{(\ell)}$). Denote its density as $P_{\bar{m}_v|\theta}^{(\ell)}$ for the considered channel $\theta \in \Upsilon$. The density $P_{\bar{m}_v|\theta}^{(\ell)}$ can be evaluated (analytically or using Monte Carlo simulation) as $\bar{m}_v^{(\ell)}$ is a function of i.i.d. messages. With respect to some optimization criterion a function $\hat{\theta}^{(\ell)}(\bar{m}_v^{(\ell)})$ is chosen. For example, maximum-likelihood (ML) criterion may be used, i.e.,

$$\hat{\theta}^{(\ell)}(\bar{m}_v) = \arg\max_{\theta \in \Upsilon} P_{\bar{m}_v|\theta}^{(\ell)}.$$

Alternatively, a criterion of minimum probability of error at the next iteration may be used,

$$\hat{\theta}^{(\ell)}(\bar{m}_v) = \arg\min_{\theta \in \Upsilon} P_{\text{error}|\theta}^{(\ell+1)}(\bar{m}_v),$$

where $P_{\text{error}|\theta}^{(\ell+1)}(\bar{m}_v)$ is the probability of error at the next iteration given $\bar{m}_v$ at the current iteration and conditioned on the channel parameter θ. Any other reasonable criterion may be used as well.

When $\hat{\theta}^{(\ell)}(\bar{m}_v)$ is decided its density, $P_{\hat{\theta}(\bar{m}_v)}^{(\ell)}$, can be evaluated by a simple change of measures.

Finally, we define the function $f_2^{(\ell)}\left(\{m_{v'e_k}^{(\ell)}, Y_{v'}\}_{v' \in R(w_k)-\{v_i\}}, Y_{v_i}\right)$ as the log-likelihood ratio transformation of the measurement Y_v assuming that the realized channel is $\hat{\theta}^{(\ell)}(\bar{m}_v^{(\ell)})$. To continue with the density evolution process, we see that obtaining the density of the LLR value, conditioned on $\theta = \hat{\theta}$, is easy (and in fact can be computed once per the entire process for each $\theta \in \Upsilon$). The true density of the output message is then simply the expectation of this conditional density over $P_{\hat{\theta}}^{(\ell)}$. The density $P_{\hat{\theta}}^{(\ell)}$ is easily obtained.

In the discussion above, we assumed that the all-zero codeword was transmitted, as some symmetry conditions similar to those in [1] were presumed to hold. However, for many compound channels the performance of the suggested decoder is not guaranteed to be independent of the transmitted codeword, mainly due to the adaptive design of the estimator function, $f_2(\cdot)$.

As a remedy, we suggest utilizing a technique originally devised by Friedlander and Burshtein [16], [30] for decoding LDPC codes over ISI channels. Here, instead of tracking a single density at each iteration, two conditional densities are to be tracked: one associated with code bits having the value 1, and the other associated with code bits having the value 0. We call the first type of density a 1-type density and the latter a 0-type density. With respect to messages passed from check nodes to variable nodes, the output density corresponds to convolutions of densities of the same type. With respect to messages passed from variable nodes to check nodes, the output density corresponds to a mixture of densities from both types, weighted by the fraction of possible combinations that yield a zero modulo-2 sum. The same holds for estimator output

message, except that the constrains associated with check equations are removed. In fact, tracking multiple densities may be extended beyond conditioning on a code bit as in [16], [30]; for example, when a complex channel is considered, densities associated with in-phase and quadrature-phase may be separately tracked.

An efficient implementation of the design process is possible by storing, for each $\theta \in \Upsilon$, only the latest distributions $P_{cv}^{(\ell-1)}$, $P_{vc}^{(\ell-1)}$ and $P_{ev}^{(\ell-1)}$ for the ℓth design iteration. If transmission of the zero-codeword cannot be assumed, twice the storage may be needed - for 0-type densities and for 1-type densities.

We also note that the approach of adapting the channel estimator to the LDPC code and to the channel[2] is in some sense dual to an approach presented in [31]. In [31] a concept of designing an LDPC code matched to a given equalizer is introduced. Also, as remarked there, both approaches may be applied to several other iterative signal processing techniques.

VI. Examples

A. Compound BSC

A compound BSC is a binary symmetric channel with an unknown transition probability, ϵ. The estimator node in this case obtains an estimate of ϵ, denoted by $\hat{\epsilon}$, and uses it to convey messages of the format $Y \ln \frac{1-\hat{\epsilon}}{\hat{\epsilon}}$. A maximum-likelihood criterion is used to evaluate $\hat{\epsilon}$. All messages are scalars.

The following proposition simplifies the density evolution process by allowing the tracking of a single density (conditioned on the all-zero codeword).

Proposition 2: The performance of a message-passing algorithm that obeys the symmetry conditions stated below is independent of the transmitted codeword if the channel obeys the symmetry condition. The proof is almost identical to the proof in [1] and is thus omitted.

The symmetry conditions referred by Proposition 2 are:

- **Channel symmetry:**
$$p(Y_k = q|X_k = 1) = p(Y_k = -q|X_k = -1).$$
- **Check node symmetry:** Considering a check node of degree k, we denote by $m_1, \ldots, m_{k-1}$ the $k-1$ input messages that yield an output message toward a variable node, v, denoted by m_{cv}. Then,
$$m_{cv}^{(\ell)}(b_1 m_1, b_2 m_2, \ldots, b_{k-1} m_{k-1}) = m_{cv}^{(\ell)}(m_1^{k-1}) \prod_{i=1}^{k-1} b_i$$
for any sequence $(b_1, \ldots, b_{k-1}) \in \{-1, 1\}^{k-1}$ and any iteration, ℓ.
- **Variable node symmetry** Considering a variable node of degree j, we denote by $m_1, \ldots, m_{j-1}$ the $j-1$ check-to-variable input messages that yield, together with the estimator-to-variable message m_0, an output message toward a check node, c, denoted by m_{vc}. Then,
$$m_{vc}^{(\ell)}(-m_0, -m_1, \ldots, -m_{j-1}) = -m_{vc}^{(\ell)}(m_0, m_1, \ldots, m_{j-1}),$$
$$m_{vc}^{(0)}(-m_0) = -m_{vc_j}^{(0)}(m_0).$$
Additionally, the variable-to-estimator message, m_{ve}, as a function of the j check-to-variable messages, $m_1, \ldots, m_j$, obeys
$$m_{ve}^{(\ell)}(-m_1, -m_2, \ldots, -m_j) = -m_{ve}^{(\ell)}(m_1, m_2, \ldots, m_j).$$

[2]Published for the first time in [15].

- **Channel estimation node symmetry:** An estimator node of degree ζ, e, is passing a message to a variable node, v_i, as a function of other variable-to-estimator messages $m_1, \ldots, m_{i-1}, m_{i+1}, \ldots, m_{zeta}$ and the measurements, $Y_1, \ldots, Y_\zeta$. Then,

$$\begin{aligned} & m_{ev_i}^{(\ell)}(b_1 m_1, b_2 m_2, \ldots, b_{i-1} m_{i-1}, b_{i+1} m_{i+1}, \ldots, b_\zeta m_\zeta; Y_1, \ldots, Y_{i-1}, Y_i, Y_{i+1}, \ldots, Y_\zeta) \\ = \;& b_i m_{ev_i}^{(\ell)}(m_1, m_2, \ldots, m_{i-1}, m_{i+1}, \ldots, m_\zeta; b_1 Y_1, \ldots, b_{i-1} Y_{i-1}, b_i Y_i, b_{i+1} Y_{i+1}, \ldots, b_\zeta Y_\zeta), \end{aligned} \quad (8)$$

  and $f_1(Y_i)$, the initial assumed prior, is an anti-symmetric function of the measurement Y_i.

The following proposition shows a rather general structure of channel estimation functions that abide by the symmetry conditions above. The proof can be found at Appendix III.

Proposition 3: Define $\forall j : \hat{m}_j \triangleq m_{v_j e} \cdot \text{sign}(Y_j)$, then a channel estimator that sends

$$m_{ev_i} = \text{sign}(Y_i) \cdot g(\hat{m}_1, \ldots, \hat{m}_{i-1}, \hat{m}_{i+1}, \ldots, \hat{m}_\zeta) \quad (9)$$

is admissible with respect to the symmetry conditions imposed above, for any function $g(\cdot)$.
Observe that from practical reasons it is possible and recommended to use the multiplications by $\text{sign}(Y_j)$ at the variable nodes, both before passing a message to the estimator node and after receiving a message from the estimator node.

Proposition 3 now allows design of transfer functions, $f_2^{(\ell)}$, by design of functions $g^{(\ell)}$, while safely relying on the all-zero codeword transmission assumption which simplifies computations.

In the example below we use the intermediate statistics $\bar{m}_v^{(\ell)} \triangleq \sum_{v' \in R(e)-\{v\}} 1\left\{\hat{m}_{v'}^{(\ell)} \geq 0\right\}$ to obtain the density $P_{\bar{m}_v|\theta}^{(\ell)}$ and determine by it the ML decision $\hat{\epsilon}$. Fig. 3 compares the ML estimation $\hat{\epsilon}(\bar{m}_v)$ with the iteration-invariant KT estimation function according to Equation (7) (note that this function obeys Proposition 3). The difference in performance is depicted in Fig. 4.

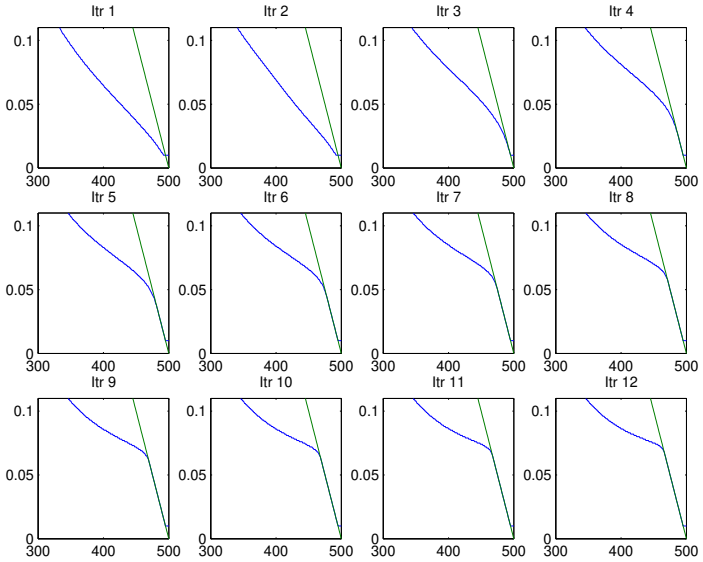


Fig. 3. The ML estimator as a function of $\bar{m}_v$, during the first 12 iterations, and the KT estimator (which is iteration invariant). Only in the high-confidence region, the ML function converges to the KT function.

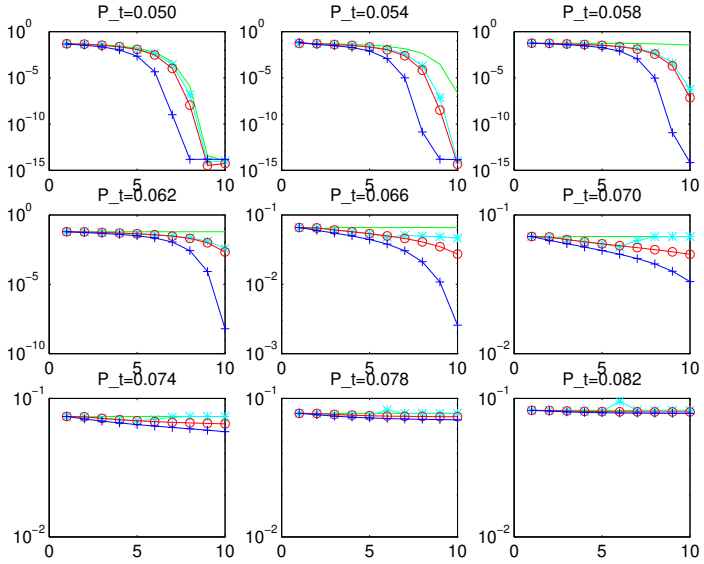


Fig. 4. The average probability of error at the first 10 iterations. With $+$ signs, the known channel is depicted. With o signs, the ML channel estimator,$\hat{\epsilon}(\bar{m}_v)$, is depicted. With $*$ signs, the ML estimator is used for the first 5 iterations, and then a switch to the KT estimator is done. The plain line describes the performance of decoding when only the KT estimator is used. At all cases, a BSC is used, and p_t stands for the true probability of transition.

B. Unknown phase complex AWGN

Consider a complex AWGN channel with a phase offset that is unknown to the receiver,

$$y = e^{j\theta}\chi + n$$

where χ, y, n are complex vectors, $j \triangleq \sqrt{-1}$ and θ is constant over a block. Write $y = y_1 + jy_2$ and the same for any other complex scalar/vector, in particular $\chi = x_1 + jx_2$.

There are two possibilities to perform decoding over such a channel (with full channel knowledge at the receiver):

1) Use a 4-ary alphabet for the messages, that corresponds to possibilities of $(1,1),(1,-1),(-1,1),(-1,-1)$ (or actually use ternary alphabet, since the sum of probabilities is 1.)
2) Use a binary alphabet, but expand the factor graph into three layers to have: check nodes, real variable nodes (x's) and complex variable nodes (χ's). Every complex variable node has a unique couple of real variable nodes. This is a viewpoint of bit-interleaved coded modulation.

Henceforth, the second approach is addressed. This is in fact a type of turbo decoding and demodulation. The update equations for the Sum-Product algorithm, which is optimal under the tree assumptions, are given at Appendix IV.

When the phase rotation is fixed but unknown, the second approach can still be used with some modifications. A three-layer graph is still sufficient here. Conceptually, a fourth layer should created, where estimator nodes are connected to complex symbol nodes. However, since a complex symbol node is connected only to a few variable nodes (2 for QPSK), it is possible to directly connect the variable nodes to the estimator nodes while absorbing the functionality of the complex-symbol nodes in the associated estimator nodes. To

retain the capabilities of the four-layer graph, the variables associated with the real part and the imaginary part of a complex symbol must both be connected to the same estimator node[3]. In accordance with the concept introduced in the previous sections, Algorithm 2 is proposed[4] for the complex AWGN channel with unknown phase rotation.

Algorithm 2:

- **Initialization:** In the absence of knowledge of the angle, θ, the algorithm rotates the phase by a quantity, τ, after which it assumes that the phase offset is corrected. Therefore, the I component (real part) is surmised to be solely associated with v_1 and the Q component (complex part) is surmised to be solely associated with v_2.

$$\begin{aligned} m_{v_1 c}^{(0)} &= m_{(y_1,y_2)v_1} = \frac{2}{\sigma^2} Re\{e^{-j\tau} y\} = \frac{2}{\sigma^2}\left[y_1 \cos(\tau) + y_2 \sin(\tau)\right] \\ m_{v_2 c}^{(0)} &= m_{(y_1,y_2)v_2} = \frac{2}{\sigma^2} Im\{e^{-j\tau} y\} = \frac{2}{\sigma^2}\left[-y_1 \sin(\tau) + y_2 \cos(\tau)\right] \end{aligned} \tag{10}$$

  It will be assumed henceforth, without loss of generality, that $\tau = 0$ as the value of $\tau \neq 0$ may be absorbed into the phase shift θ.

- **Check-to-Variable:**

$$\tanh\left(\frac{m_{cv}^{(\ell)}}{2}\right) = \prod_{v' \in L(c)-\{v\}} \tanh\left(\frac{m_{v'c}^{(\ell-1)}}{2}\right)$$

- **Variable-to-Estimator:**

$$m_{ve}^{(\ell)} = \sum_{c' \in R(v)} m_{c'v}^{(\ell)}$$

- **Estimator-to-Variable:**

$$\hat{M}_{v_i'} = \text{sign}(m_{v_i' e}^{(\ell)}) \;\; i \in \{1,2\}, v' \in R(e) - \{v\}$$

$$a_{v_1'} = \frac{1}{2} y_{v_1'} \hat{M}_{v_1'} + \frac{1}{2} y_{v_2'} \hat{M}_{v_2'}$$

$$a_{v_2'} = -\frac{1}{2} y_{v_1'} \hat{M}_{v_2'} + \frac{1}{2} y_{v_2'} \hat{M}_{v_1'}$$

$$\tilde{A}_i = \frac{1}{k} \sum_{v' \in R(e)-\{v\}} a_{v_i'}^r \;\; i \in \{1,2\}$$

  Calculate $\hat{\theta}^\ell = f_2^{(\ell)}(\tilde{A}_1, \tilde{A}_2)$, where $f_2^{(\ell)}(\cdot)$ may be a function designed by means of density evolution as proposed in Section V.

$$\begin{aligned} m_{ev_1}^{(\ell)} &= \frac{2}{\sigma^2} Re\{e^{-j\hat{\theta}^{(\ell)}} y\} \\ m_{ev_2}^{(\ell)} &= \frac{2}{\sigma^2} Im\{e^{-j\hat{\theta}^{(\ell)}} y\} \end{aligned} \tag{11}$$

[3]A practical way to do it is to use even degree estimator nodes and to declare the I/Q identities only as a last step.

[4]This algorithm is not optimized, for example, with respect to the statistics in use. In particular, the variable-to-estimator messages go through hard-decision that should incur some loss of performance.

- **Variable-to-Check:**

$$m_{vc}^{(\ell)} = m_{ev}^{(\ell)} + \sum_{c' \in R(v)-\{c\}} m_{c'v}^{(\ell)}$$

- **Termination:**

$$m_{v}^{(\ell)} = m_{ev}^{(\ell)} + \sum_{c' \in R(v)} m_{c'v}^{(\ell)}$$

It is easy to see that, in general, the algorithm is not independent of the transmitted codeword. Observe the real component in Equation (11), its first part expended as Equation (12).

$$m_{ev_1}^{(\ell)} = \frac{2}{\sigma^2}[x_1 \cos(\theta - \hat{\theta}^{(\ell)}) - x_2 \sin(\theta - \hat{\theta}^{(\ell)}) + \tilde{n}_1]. \tag{12}$$

Not only a flip in $m_{ev_1}^{(\ell)}$ is not assured by a flip in x_1 (e.g., consider $\theta - \hat{\theta}^{(\ell)} = \frac{\pi}{2}$), but also a change of sign

$$\text{sign}[\cos(\theta - \hat{\theta}^{(\ell-1)})] \neq \text{sign}[\cos(\theta - \hat{\theta}^{(\ell)})]$$

may cause a sign change in $m_{ev_1}^{(\ell)}$. As suggested in Section V, this analysis problem is solved by addressing two conditional densities - a 1-type density and a 0-type density. In fact, for the analysis of Algorithm 2 another type of conditioning is also required for certain steps - conditioning on the in-phase/quadrature-phase elements.

We elaborate now on the density evolution process to demonstrate the interactions between the tracked conditional densities. The complexity of the analysis, which deals with these interactions and also with two-dimensional densities, is also discussed. For brevity sake, a regular ensemble is assumed, having variable, check and estimator degrees of d_v, d_c and d_e respectively.

In a Gray mapping, the real component (I) is associated with the variable previously named x_1, and the imaginary component (Q) with x_2. We will use both terms. Additionally, the variables x_1 and x_2 are associated with variable nodes v_1 and v_2 respectively. A type-1 density is associated with a BPSK symbol $x = -1$ and a type-0 density is associated with a BPSK symbol $x = +1$.

The initial message for v_1, has the conditioned p.d.f

$$\begin{aligned} P_{m_{yv_1}|x_1=\pm1}(\alpha) &= P_{\frac{2}{\sigma^2}Re\{e^{-j\tau}y\}|x_1=\pm1}(\alpha) = P_{\frac{2}{\sigma^2}Re\{\chi e^{j(\theta-\tau)}+ne^{-j\tau}\}|x_1=\pm1}(\alpha) \\ &= \sum_{x_2\in\{\pm1\}} \frac{1}{2} P_{\frac{2}{\sigma^2}[x_1\cos(\theta-\tau)-x_2\sin(\theta-\tau)+\tilde{n}_1]|x_1=\pm1,x_2}(\alpha) \\ &= \frac{1}{2} P_{\frac{2}{\sigma^2}[\pm\cos(\theta-\tau)-\sin(\theta-\tau)+\tilde{n}_1]}(\alpha) + \frac{1}{2} P_{\frac{2}{\sigma^2}[\pm\cos(\theta-\tau)+\sin(\theta-\tau)+\tilde{n}_1]}(\alpha), \end{aligned} \tag{13}$$

where $\tilde{n} = e^{-j\tau}n$ has the same statistic characterization of the complex scalar n.

Similarly, for v_2, the p.d.f is given by

$$\begin{aligned} P_{m_{yv_2}|x_2=\pm1}(\alpha) &= P_{\frac{2}{\sigma^2}Im\{e^{-j\tau}y\}|x_2=\pm1}(\alpha) = P_{\frac{2}{\sigma^2}Im\{\chi e^{j(\theta-\tau)}+ne^{-j\tau}\}|x_2=\pm1}(\alpha) \\ &= \sum_{x_1\in\{\pm1\}} \frac{1}{2} P_{\frac{2}{\sigma^2}[x_1\sin(\theta-\tau)+x_2\cos(\theta-\tau)+\tilde{n}_2]|x_2=\pm1,x_1}(\alpha) \\ &= \frac{1}{2} P_{\frac{2}{\sigma^2}[\sin(\theta-\tau)\pm\cos(\theta-\tau)+\tilde{n}_2]}(\alpha) + \frac{1}{2} P_{\frac{2}{\sigma^2}[-\sin(\theta-\tau)\pm\cos(\theta-\tau)+\tilde{n}_2]}(\alpha) \end{aligned} \tag{14}$$

It is not too hard to see that $P_{m_{yv_1}|x_1=\pm 1}(\alpha) = P_{m_{yv_2}|x_2=\pm 1}(\alpha)$ and that this density is the density of a mixture of two Gaussians. Note that in the Estimator-to-Variable massaging stage, the parameter $\hat{\theta}$ replaces the initial rotation, τ. However, as for any τ the marginal I/Q distributions are the same, it holds for a any $\hat{\theta}$ as well. Note that there is no need to go beyond the marginal distributions into joint distributions due to the tree structured neighborhood assumption.

Let $\Phi(\underbrace{p_{m_{v^0c}},\ldots}_{d_c-1-t},\underbrace{p_{m_{v^1c}},\ldots}_{t})$ be the density of the outgoing Check-to-Variable message, assuming that t of the incoming messages are of type-'1' density and d_c-1-t messages are of type-'0' density. Then,

$$p_{m_{cv}} = \sum_{t=0}^{d_c-1} \Phi(\underbrace{p_{m_{v^0c}},\ldots}_{d_c-1-t},\underbrace{p_{m_{v^1c}},\ldots}_{t}) P(t,d_c)$$

where $P(t,d_c)$ is the probability that t input messages come indeed from a type-1 variable node. The parity-check constraint of the code dictates that if the variable node v is associated with a type-0 density (i.e., a bit $x=1$) then t must be even, (hence $P(t,d_c)=0$ for an odd value of t). If v is associated with a type-1 density then t must be odd. Again, the output density is type-'0/1' dependent, but not I/Q-dependent.

The simulation results obtained in this paper presumed that

$$P(t,d_c) = \begin{cases} \dfrac{\binom{d_c-1}{t}}{\sum_{t'\in T}\binom{d_c-1}{t'}} & t \in T \\ 0 & t \notin T \end{cases} \tag{15}$$

where T is the set of even (resp. odd) numbers less than or equal to d_c-1 for a type-0 (resp. type-1) variable destiny. In fact, Equation (15) assumes that all d_c-1 bits that participates in a certain parity-check constraint are independent and of equal probability. If the minimum check node degree is exactly the minimum distance of the dual code (in general it is an upper bound to the distance), then Theorem 8 of [32, Ch. 5, Sec. 5][5] can be used to guarantee that indeed the d_c-1 inputs are independent and equi-probable. However, for irregular codes this approach does not apply. Luckily, as shown by Wang *et al* [33, Th. 3], the number of nodes whose input messages fail to obey this condition vanishes with the block length approaching infinity (i.e., the inputs approach i.i.d. in probability).

The variable-to-Estimator message density, $p_{m_{ve}}$, is given by d_v convolutions of $p_{m_{cv}}$. The output is a bit-type dependent density, but not I/Q-dependent. Following we get

$$p_{\hat{M}} = \left(\int_{-\infty}^{0} p_{m_{ve}}(\alpha)d\alpha, \int_{0}^{\infty} p_{m_{ve}}(\alpha)d\alpha\right)$$

The direct method of obtaining the joint 2-dimensional density $P_{\tilde{A}_1,\tilde{A}_2}$ is memory consuming (see Appendix V). The following method has significantly less storage complexity.

The complex variable a can be written as

$$a = \left(e^{j\theta}\sqrt{2}e^{j\frac{\pi}{2}(s+\frac{1}{2})} + n\right)\frac{\sqrt{2}}{2}e^{-j\frac{\pi}{2}(m+\frac{1}{2})}$$

[5]Denote by $d^{\perp}$ the minimum distance of the dual code. The theorem says that any vector of r code symbols is uniformly distributed (over a discrete set) for $r \le d^{\perp}-1$, and $d^{\perp}$ is the largest number with this property. Note that if a vector is uniformly distributed, then its marginals are i.i.d. and equi-probable.

where s designates the transmitted QPSK signal and m designates the estimated QPSK signal (corresponding to $\hat{M}$).

Lemma 2: The random variable a has the same distribution as a' defined by

$$a' = e^{j\theta}e^{j\frac{\pi}{2}(s-m)} + \frac{1}{\sqrt{2}}n \triangleq e^{j\theta}B + \frac{1}{\sqrt{2}}n$$

Proof: The noise, n, is circular-symmetric Gaussian and is independent of m by the tree assumption and since the channel is memoryless. For any $m \in \{1,2,3,4\}$ the variable $e^{-j\frac{\pi}{2}\left(m+\frac{1}{2}\right)}n$ is identically distributed as n, and moreover it is independent of m (i.e., B and n are independent). ■

The random variable, B, is a complex Bernoulli variable in $\{\pm 1, \pm j\}$, which is equal to $\frac{1}{2}\hat{M}^{\dagger}\cdot\chi$, where $\hat{M}^{\dagger}$ is the complex conjugate of $\hat{M}$. It is quite simple to calculate the distribution of B from the conditional distributions of $\hat{M}$. The variables a and B are not bit-type dependent, as they serve only for estimation of θ. This is the point of the analysis where the bit-type conditional densities are converted into I/Q-dependent densities that are bit-type independent. In fact, 2-dimensional densities must be kept to hold the mutual relationship between the I and the Q components.

The average over k such variables (i.e., the variable $\tilde{A}$) is distributed as A' does, where

$$A' = e^{j\theta}\frac{1}{k}\sum_{i=1}^{k}B_i + n'$$

where $n' \sim N\left(0, \frac{1}{2k}\sigma^2 I_{2\times 2}\right)$.

It is not difficult to compute the (2-dimensional) distribution of the complex random variable (having a complex Binomial distribution), $\sum_{i=1}^{k}B_i$, by the assistance of a two-dimensional FFT transformation. Note that with respect to storage complexity, the matrix size containing the convolution's output is no larger than $2k$.

Once the distribution of $\sum_{i=1}^{k}B_i$ is given, it is easy to obtain the (2-dimensional) distribution of A' by first rotating with respect to θ and then accounting for the addition of the complex circular-symmetric Gaussian noise.

Then, the distribution of the real variable, $\hat{\theta}$, is obtained by a change of measures; this density is neither bit-type dependent nor I/Q dependent. Note that $\tilde{A}$ may receive values off the unit circle, and the estimator function, $f_2(\cdot)$, must be designed to take care of this situation. Now, the expectation with respect to the density of $\hat{\theta}$ is taken according to

$$P_{ev_1|x_1}(\alpha) = \int_{-\pi}^{\pi} d\tau P_{\hat{\theta}}(\tau)\frac{1}{2}\left[P_{\frac{2}{\sigma^2}[x_1\cos(\theta-\tau)-\sin(\theta-\tau)+\tilde{n}_1]}(\alpha) + P_{\frac{2}{\sigma^2}[x_1\cos(\theta-\tau)+\sin(\theta-\tau)+\tilde{n}_1]}(\alpha)\right] \tag{16}$$

To complete the round, the distribution of the messages $p_{m_{v^{I_c}}}$ and $p_{m_{v^{Q_c}}}$ are now found by (real) convolutions, with respect to their bit type.

The density evolution process is computationally more demanding when 2-dimensional densities are considered. It is nice to show that the number of operations can be reduced by addressing only angles in $[0,\pi]$. Proposition 4 shows that indeed the design process may be carried over $\theta \in [0,\pi]$ by using

$$f_2(\tilde{A}) = \text{sign}(\arg(\tilde{A}))f_3\left(|\tilde{A}|e^{j|\arg(\tilde{A})|}\right)$$

where $\tilde{A} = \tilde{A}_1 + j\tilde{A}_2$ and $f_3(\cdot)$ is the function designed by the process at the relevant iteration over the grid of positive part of the phase shift grid.

Proposition 4: The maximum-likelihood map evaluated by the above density evolution process, $\hat{\theta}_{ML}$, is anti-symmetric as a function of θ, if the initial phase rotation is $\tau = 0$.

Proof: For a start, it is clear from Equations (13)-(14) that for $\tau = 0$, the measurement-to-variable densities are symmetric with respect to θ. That is

$$P_{m_{yv}}^{\theta=\theta'} = P_{m_{yv}}^{\theta=-\theta'}$$

This of course implies that the distributions of B and $\beta \triangleq \frac{1}{k}\sum_i B_i$ are symmetric with respect to θ. Note that A' is distributed as A'' does:

$$A'' = e^{j\theta}\left(\beta + n'\right)$$

Thus, by construction, for any complex scalar α,

$$\begin{aligned} P_{A''}^{\theta=\theta'}\left(\alpha e^{j\theta'}\right) &= P_{A''}^{\theta=\theta'}\left(\alpha e^{j\theta}\right) = \int P_{\beta}^{\theta=\theta'}(\nu) P_{n'}^{\theta=\theta'}(\alpha-\nu)d\nu \\ &= \int P_{\beta}^{\theta=-\theta'}(\nu) P_{n'}^{\theta=-\theta'}(\alpha-\nu)d\nu \\ &= P_{A''}^{\theta=-\theta'}\left(\alpha e^{j\theta}\right) = P_{A''}^{\theta=-\theta'}\left(\alpha e^{-j\theta'}\right) \end{aligned}$$

and by $P_{\tilde{A}}^{\theta=\theta'} = P_{A''}^{\theta=\theta'}$

$$\forall \alpha:\ P_{\tilde{A}}^{\theta=\theta'}\left(\alpha e^{j\theta'}\right) = P_{\tilde{A}}^{\theta=-\theta'}\left(\alpha e^{-j\theta'}\right) \tag{17}$$

The maximum-likelihood map (and many other types of maps too) thus obeys

$$\forall \alpha:\ \hat{\theta}\left(\alpha e^{j\theta}\right) = -\hat{\theta}\left(\alpha e^{-j\theta}\right)$$

which means that $\hat{\theta}$ is an anti-symmetric function of θ.

The density in Equation (11), is the expectations of densities like in Equations (13)-(14). As the integrands are symmetric with respect to θ we conclude again that P_{ev_1} and P_{ev_2} are symmetric with respect to θ. This guarantees that $P_B^{\theta=\theta'} = P_B^{\theta=-\theta'}$ for any iteration, which maintains the property that $\hat{\theta}(\theta) = -\hat{\theta}(-\theta)$ as claimed. ■

Proposition 4, using the property given by Equation (17), allows cutting in half the number of computations needed as only half the grid, $\{\theta \in \Upsilon : \theta \geq 0\}$, must be used for the density evolution process that is required for the design of the estimator transfer function.

For the ensemble $\lambda_2 = 0.37, \lambda_4 = 0.63, \rho_5 = 0.5, \rho_7 = 0.5, \eta_{20} = 1.0$ (code rate approximately 0.5) and the phase rotation grid $\Upsilon = \{-87^0, -84^0, \ldots, 84^0, 87^0\}$ Figures 5-7 show the response to different phases as a function of the number of iteration. As can be seen, monotonicity with respect to bit error probability may be verified[6]. The threshold as a function of $\frac{E_b}{N_0}$ is depicted in Fig. 8. It turns out that a wider phase grid may compromise the phase threshold function. For example, in the conditions depicted, at $E_b/N_0 = 2dB$ the threshold is around 33^0 when the grid is tested between -45^0 and 45^0, while the threshold is around 30^0 when the tested grid is extended to -87^0 to 87^0. The results in this figure are slightly better than the results obtained by [6]. Additionally to the two curves associated with Algorithm 2, two curves are shown as a reference. The first is the performance of a decoder that simply assumes no phase rotation; for this

[6]When the iteration number is big, the cross-over between curves might be due to precision limitations, mainly in the expensive resource that corresponds to the 2-dimensional distributions storage and the density rotation operations.

decoder a loss of approximately 1dB to 2dB is observed for phase offsets above 15^0. The second curve is an upper bound on the performance under known channel conditions (no phase rotation), but considering a power reduction due to the smaller projection over the axes (the interference by projection on the wrong axis is taken into account in the previous curve, but not in this curve). It is easy to show that the power is reduced by a factor of $\cos^2(\theta)$, so the threshold bound curve is given by

$$\arccos\left(\sqrt{\left(\frac{E_b}{N_0}\right)_{thr} - \frac{E_b}{N_0}}\right),$$

where the known channel threshold of the exemplified ensemble is at $E_b/N_0 = 0.82$dB.

Finally, a natural generalization of the unknown phase approach is to include also an unknown noise variance into the grid of examined parameters, Υ. Thus, block fading is immediately dealt with the tools introduced above. Also, channel models for which the parameter is fixed only during short sub-blocks (as in [8]) can be treated using the tools introduced in this paper. Thus, for example, for frequency shift uncertainty many phase estimator nodes can do the work. Alternatively, for frequency shift uncertainty, many frequency shift estimators can be used, although the shift may be fixed throughout the block (as the phase was fixed in the above example).

VII. Conclusions

A scheme for iterative joint decoding and channel estimation was proposed, suitable to many classes of channels. The scheme suggests a non-standard approach for the connection of the channel estimation stage and the decoding stage, allowing for rigorous numerical analysis of the bit error probability of the entire scheme by density evolution. Stemming out of this new capability is a concept of design of channel estimators that are adapted to the specific LDPC code ensemble in use, the number of iteration and the considered compound channel. The solution presented operates under a non-Baysian criterion, i.e., no distribution is assumed on the parameter that rules the realization of the channel. Numerical results show small gains over advanced techniques known in the literature for the same purpose, even for a non-optimized ensemble, and emphasize the importance of the selection of channel estimator.

Acknowledgements

We wish to thank J. Ziv for insightful discussions in the preliminary phases of this research.

Appendix I
Proof of Lemma 1

We will now show that a tree property holds for a more useful scenario of random interleaving at the "right" part of the graph and a deterministic fixed interleaving at the "left" part of the graph. This scenario helps addressing physical-like issues, such as constant channel parameters over short sub-blocks (as in frequency hopping, for example) or an approximation of a slowly varying channel (which is a channel with memory).

To show the tree property, consider "mega-nodes", where each mega-node corresponds to a unique estimator node. The mega-node absorbs into it all the variable nodes attached to the corresponding estimator node. Thus, the number of sockets associated with this mega-node is the sum of the number of sockets of each variable node attached to the corresponding estimator node. For simplicity, we assume henceforth that

our graph is regular, so a mega-node has $d_v d_e$ sockets. The interleaving between the "left" $e \cdot d_v d_e$ sockets and the "right" $m \cdot d_c$ sockets, where m is the number of check nodes, is the exact uniform interleaving between the nd_v variable nodes' sockets and md_c check node sockets. Thus, from [1, Appendix A] we conclude that the probability of a depth ℓ neighborhood in the new graph is lower bounded by

$$\left(1-\frac{M_\ell}{e}\right)^{M_\ell}\left(1-\frac{C_\ell}{m}\right)^{C_\ell} \geq 1-\frac{M_\ell^2}{e}-\frac{C_\ell^2}{m}$$

where the variables M_ℓ and C_ℓ denote the number of "left" nodes (mega-nodes) and "right" nodes (check nodes) existing in a tree-like neighborhood of depth ℓ. Also, from [1, Appendix A], these values obey $M_\ell = \mathcal{O}((d_e d_v)^\ell d_c^\ell)$ and $C_\ell = \mathcal{O}((d_e d_v)^\ell d_c^{\ell-1})$. We also have $e = \frac{n}{d_e}$ (or in an irregular graph, e is lower bounded by n over the maximal estimator node degree). Hence we conclude that a sufficient condition for convergence to a tree-like neighborhood in probability is that d_e grows sub-polynomially with n. For example, $d_e = \mathcal{O}(\log(n))$ or d_e =constant.

Until now we have shown that the graph containing the mega-nodes is cycle-free up to any fixed depth, in probability. Since the sub-graph hidden inside a mega-node (one estimator node connected to several variable nodes) is itself a tree, we can safely conclude that the three-layered graph has the tree property as well. ■

APPENDIX II
PROOF OF PROPOSITION 1

We illuminate only the minor changes required to the proof of [1, Theorem 2] to hold also for Proposition 1. Other than the changes mentioned below, the proofs are identical.

- **Convergence to Cycle-Free Case:** The proof relies on the tree property. The directed neighborhood to be considered at the ℓth iteration corresponds to a propagation depth of 4ℓ edges and not 2ℓ as in [1].
- **Concentration Around Cycle-Free Case:** No changes.
- **Concentration Around Expected Value:** The edge exposure, which precedes the vertex exposure, contains also exposure of estimator-to-variable edges. The random variables considered still form a Doob's martingale process. The difference sequence is still bounded by terms associates only with the node degrees and the iteration number, and not with the block length; however, the bounding constants may be different.

■

APPENDIX III
PROOF OF PROPOSITION 3

We have to show that the function in (9) obeys that channel estimation node symmetry condition in (8). For any arbitrary $(b_1,\ldots,b_\zeta) \in \{-1,1\}^\zeta$:

$$\begin{aligned} & m_{ev_i}^{(\ell)}\left(\{b_j m_j\}_{j=1}^{i-1},\{b_j m_j\}_{j=i+1}^{\zeta};\{Y_j\}_{j=1}^{\zeta}\right) \\ &= \text{sign}(Y_i)\cdot g\left(\{b_j m_j \cdot \text{sign}(Y_j)\}_{j\neq i}\right) \\ &= b_i \text{sign}(b_i Y_i)\cdot g\left(\{m_j \cdot \text{sign}(b_j Y_j)\}_{j\neq i}\right) \\ &= b_i m_{ev_i}^{(\ell)}\left(\{m_j\}_{j=1}^{i-1},\{m_j\}_{j=i+1}^{\zeta};\{b_j Y_j\}_{j=1}^{\zeta}\right). \end{aligned}$$

■

APPENDIX IV
UPDATE EQUATIONS FOR THE SUM-PRODUCT OVER THE KNOWN COMPLEX AWGN CHANNEL

Let v be a real variable node, and denote by $\tilde{v}$ its sibling real variable node such that v and $\tilde{v}$ belong to the same complex variable node χ.

Define S_c as the event that the constraint associated with a check node c is satisfied. Define $S_{x_i c} \triangleq \bigcap_{c' \in R(x_i)-\{c\}} S_{c'}$, that is the event that all checks connected to x_i, except for c, are satisfied. Define S_{x_i} as the event that all check nodes connected to x_i are satisfied, that is $S_{x_i} = S_{x_i c} \cap S_c$.
Define Y_{cv} as the set of measurements $\{y_i\}$ associated with the variable nodes in the directed neighborhood of check node c excluding the neighborhood of variable node v, which is connected to c. Define $Y_v \triangleq (y_1, y_2) \cup \bigcup_{c \in R(v)} Y_{cv}$, where (y_1, y_2) are the measurements associated with v itself.

Let

$$m_{cv} = \log \frac{P(S_c|v=1, Y_{cv})}{P(S_c|v=-1, Y_{cv})}$$

$$m_{vc} = \log \frac{P(v=1|S_{vc}, S_{\tilde{v}}, Y_v)}{P(v=-1|S_{vc}, S_{\tilde{v}}, Y_v)}$$

$$m_{v\chi} = \log \frac{P(S_v|v=1, Y_v)}{P(S_v|v=-1, Y_v)}$$

$$m_{\chi v} = \log \frac{P(v=1|S_{\tilde{v}}, Y_{\tilde{v}})}{P(v=-1|S_{\tilde{v}}, Y_{\tilde{v}})}$$

$$\begin{aligned} m_{(y_1,y_2)v} &= \log \frac{P(v=1|y_1, y_2)}{P(v=-1|y_1, y_2)} \\ &= \log \frac{\sum_{\tilde{v} \in \{\pm 1\}} P(v=1, \tilde{v}|y_1, y_2)}{\sum_{\tilde{v} \in \{\pm 1\}} P(v=-1, \tilde{v}|y_1, y_2)} \end{aligned}$$

Assuming all codeword are equi-probable, the relations between the definitions above translate into the following update equations of the Sum-Product:

- **Initial massage:**

$$m_{vc}^{(0)} = m_{(y_1,y_2)v}$$

- **Check-to-Real variable:**

$$\tanh\left(\frac{m_{cv}^{(\ell)}}{2}\right) = \prod_{v' \in L_c - \{v\}} \tanh\left(\frac{m_{v'c}^{(\ell-1)}}{2}\right)$$

- **Real variable-to-Complex variable:**

$$m_{v\chi}^{(\ell)} = \sum_{c' \in R(v)} m_{c'v}^{(\ell)}$$

- **Complex variable-to-Real variable:**

$$m_{\chi v} = \log \frac{P(y_1, y_2|v=1, \tilde{v}=1)e^{m_{\tilde{v}\chi}} + P(y_1, y_2|v=1, \tilde{v}=-1)}{P(y_1, y_2|v=-1, \tilde{v}=1)e^{m_{\tilde{v}\chi}} + P(y_1, y_2|v=-1, \tilde{v}=-1)}$$

- **Real variable-to-Check:**
$$m_{vc}^{(\ell)} = m_{\chi v}^{(\ell)} + \sum_{c' \in R(v)-\{c\}} m_{c'v}^{(\ell)}$$
- **Termination:**
$$m_{v}^{(\ell)} = m_{\chi v}^{(\ell)} + \sum_{c' \in R(v)} m_{c'v}^{(\ell)}$$

If $P(y_1, y_2|v, \tilde{v}) = P(y_1|v)P(y_2|\tilde{v})$ then $m_{\chi v} = m_{(y_1,y_2)v}$. Examples in which $P(y_1, y_2|v, \tilde{v})$ is *not separable* are

- QPSK modulation with the mapping:
$$\begin{array}{ccc} (0,0) & \mapsto & (1,1) \\ (0,1) & \mapsto & (1,-1) \\ (1,0) & \mapsto & (-1,-1) \\ (1,1) & \mapsto & (-1,1) \end{array}$$
- 8-PSK modulation scheme, etc.
- QPSK modulation with Gray mapping (or any other mapping[7]) but with unknown phase translation.

APPENDIX V
COMPLEXITY

With respect to phase estimation over the complex AWGN, the following approach, which is theoretically valid, requires storage complexity that is basically unpractical.

We need to use the *joint* density of a^I and a^Q, to be able to use $(\tilde{A}^I, \tilde{A}^Q)$ afterwards. This is easily done by computing

$$p_{a^I,a^Q} = \sum_{v_1,v_2} \frac{1}{4} p_{a^I,a^Q|v_1,v_2} = \sum_{v_1,v_2} \frac{1}{4} \sum_{m_1,m_2 \in \{\pm 1\}} p_{\hat{M}|v_1}(m_1) p_{\hat{M}|v_2}(m_2) p_{a^I,a^Q|m_1,m_2,v_1,v_2}$$

Given (v_1, v_2), the distribution of a_1, a_2 depends on the types of the variable bits (whether they are ± 1). However, for the variable node that is the destination of this estimate the types of the bits that derive this estimate are irrelevant. Therefore we average over the 4 possible conditional densities at this point. Unlike the averaging at the check nodes, the estimator nodes impose no restriction on the type of the variable nodes attached to them. Hence, each of the four densities receives a weight of $1/4$. Note that the averaging cannot be moved backward to $\hat{M}$, as a depends on both $\hat{M}$ and y, and the latter is even more heavily dependent on the sign of v_i.

Notice that given $\hat{M}_1, \hat{M}_2, v_1, v_2$, the random variables a_1 and a_2 are jointly Gaussian. Moreover, whenever one of them is the sum of two Gaussian variables, the other is their difference. This implies that under these terms the two variables $a_1|\hat{M}_1, \hat{M}_2, v_1, v_2$ and $a_2|\hat{M}_1, \hat{M}_2, v_1, v_2$ are independent! Therefore,

$$p_{a^I,a^Q|v_1,v_2} = \sum_{m_1,m_2 \in \{\pm 1\}} p_{\hat{M}|v_1}(m_1) p_{\hat{M}|v_2}(m_2) p_{a^I|m_1,m_2,v_1,v_2} p_{a^Q|m_1,m_2,v_1,v_2}$$

[7]Note that Gray mapping results in a separable probability function for a known phase.

Consider, for example, the density

$$p_{a^I|m_1=1,m_2=1,v_1,v_2} = P\left(\frac{y_1+y_2}{2}\middle| v_1, v_2\right)$$

and the conditioning by m_1, m_2 can be dropped out (they are a function of extrinsic information only). Such a density is calculated and stored once in the beginning of the process; it resembles the initial message density with $\tau = 45^\circ$ and a different normalization term. In fact, the reason for this density to be unchanged throughout the process is because this is where measurements mix with process-dependent messages, and this density corresponds to the measurements part. Note that

$$P_{\frac{y_1+y_2}{2}|v_1,v_2}(\alpha) = P_{\frac{1}{2\sqrt{2}}\left[v_1\cos(\theta+\frac{\pi}{4})-v_2\sin(\theta+\frac{\pi}{4})+\tilde{n}_1\right]|v_1,v_2}(\alpha)$$

and

$$P_{\frac{y_1-y_2}{2}|v_1,v_2}(\alpha) = P_{\frac{1}{2\sqrt{2}}\left[v_1\cos(\theta-\frac{\pi}{4})-v_2\sin(\theta-\frac{\pi}{4})+\tilde{n}_1\right]|v_1,v_2}(\alpha) = P_{\frac{1}{2\sqrt{2}}\left[v_1\sin(\theta+\frac{\pi}{4})+v_2\cos(\theta+\frac{\pi}{4})+\tilde{n}_1\right]|v_1,v_2}(\alpha)$$

and only two such densities should be stored, $P_{y_1+y_2|x_1=1,x_2=1}(\alpha)$ and $P_{y_1-y_2|x_1=1,x_2=1}(\alpha)$. A distribution such as $P_{y_1+y_2|x_1=-1,x_2=1}(\alpha)$ is equal to $P_{y_1-y_2|x_1=-1,x_2=-1}(\alpha) = P_{y_1-y_2|x_1=1,x_2=1}(-\alpha)$. Similar rules give all other needed densities. The notation $\tilde{n}_1$ is used here to denote a real $N(0,\sigma^2)$ Gaussian variable.

The distribution of $\tilde{A}$ is obtained by complex convolutions, that is of the type

$$P(X+Y=a+ib) = \sum_{c,d} P(X=c+id)P(Y=(a-c)+i(b-d))$$

followed by normalization stage, i.e., for quantized distribution

$$P_{\frac{A}{k}}(\alpha) = \sum_{\beta:\left[\frac{A}{k}\right]=\beta} P_A(\beta)$$

Note that there is only one type of distribution here (though over a complex domain, this time due to I/Q and not the variable node type).

Assume that averaging is of k elements, then the convolution operation (not cyclic convolution) increases the size of the database k times in each dimension. Consider, e.g., a density represented by 1000×1000 points. For $k=20$ (which cannot be considered too large) the convolution's output contains at least $4\cdot 10^8$ points (or if powers of 2 are required for FFT implementation, we get $32768\times 32768 = 2^{30}$ which is about 1 Giga-points (thus, 8 GB are required!).

As a better solution, we can suggest the following method of averaging over k elements. We will explain it by an example. For a given k, define $m = \left\lfloor\frac{k}{2}\right\rfloor$ and $\ell = \left\lceil\frac{k}{2}\right\rceil$. Now,

$$\frac{1}{k}\sum_{i=1}^{k} a_i = \frac{\ell}{k}\left(\frac{1}{\ell}\sum_{i=1}^{\ell} a_i\right) + \frac{m}{k}\left(\frac{1}{m}\sum_{i=1}^{m} a_{\ell+i}\right)$$

So we average of ℓ elements and over m elements, and then average over the two results. Of course, each "smaller" convolution may be recursively computed.

The advantage of this method is that average distributions are typically contained in a matrix of the same size of the matrices associated with the elements themselves (i.e., no expansion of the matrix size). Let $N\times N$ denote the size of an element's distribution matrix, then instead of using once a matrix of size k^2N^2 we use $\sim \log_2 k$ levels which provides us $\sim k\log_2 k$ convolutions of size 2^2N^2.

The disadvantage of this method is that the weighting that needs to take place before averaging two components is a lossy operation when discrete distributions are used.

Another, even better method is by decomposing k to radix 2. For example, if $k = 19$ then we can compute the average over 16 elements, 2 elements and 1 element ($16+2+1=19$), each by exact recursive computation that is nearly lossless; then, still a weighted average is required ($\frac{16}{19}, \frac{2}{19}, \frac{1}{19}$), but less times. Here we need not $k \log_2 k$ convolutions but only $\log_2 k$ convolutions of size $2^2 N^2$.

REFERENCES

[1] T. Richanrdson and R. Urbanke, "The capacity of low-density parity check codes under message-passing decoding," *IEEE Trans. on Information Theory*, vol. 47, no. 2, pp. 599–618, Feb. 2001.

[2] M. Valenti and B. Woerner, "Iterative channel estimation and decoding of pilot symbol assisted turbo codes over flat-fading channels," *IEEE Journal on Sel. Areas in Comm.*, vol. 19, no. 9, pp. 1697–1705, Sep. 2001.

[3] C. Komninakis and R. Wesel, "Joint iterative channel estimation and decoding in flat correlated rayleigh fading," *IEEE Journal on Sel. Areas in Comm.*, vol. 19, no. 9, pp. 1706–1717, Sep. 2001.

[4] T. Minowa and H. Imai, "Robustness of iterative principles to channel estimation mismatch," in *Proc. of ISIT 2002*, 2002, p. 469.

[5] X. Wang and R. Chen, "Blind turbo equalization in gaussian and impulsive noise," *IEEE Trans. on Vehicular Technology*, vol. 50, no. 4, pp. 1092–1105, July 2001.

[6] W. Oh and K. Cheun, "Joint decoding and carrier phase recovery algorith for turbo codes," *IEEE Communications Letters*, vol. 5, no. 9, pp. 375–377, Sep 2001.

[7] R. Nuriyev and A. Anastasopoulos, "Analysis of joint iterative decoding and phase estimation for the noncoherent awgn channel, using density evolution," in *Proc. of ISIT 2002*, 2002, p. 168.

[8] ——, "Pilot-symbol-assisted coded transmission over the block-noncoherent awgn channel," *IEEE trans. on Communications*, vol. 51, no. 6, June 2003.

[9] J. Dauwels and H.-A. Loeliger, "Joint decoding and carrier synchronization using factor graphs," in *Winter School on Coding and Inform. Theory*, Monte Verita, Switzerland, Feb. 2003.

[10] ——, "Joint decoding and phase estimation: An exercise in factor graphs," in *Proc. of ISIT 2003*, Yokahama, Japan, July 2003, p. 231.

[11] A. Lapidoth and P. Narayan, "Reliable communication under channel uncertainty," *IEEE Trans. on Information Theory*, vol. 44, no. 6, pp. 2148–2177, Oct. 1998.

[12] A. Burr and L. Zhang, "Iterative joint synchronisation and turbo-decoding," in *Proc. of ISIT 2002*, Lausanne, Switzerland, 30 June - 5 July 2002, p. 414.

[13] H. Steendam, N. Noels, and M. Moeneclaey, "Iterative carrier phase synchronization for low-density parity-check coded systems," in *Proc. of ICC 2003*, vol. 5, 2003, pp. 3120–3124.

[14] W. Matsumoto and H. Imai, "Blind synchronization with enhanced sum-product algorithm for low-density parity-check codes," in *Proc. of International Symposium on Wireless Personal Multimedia Communications*, vol. 3, Honolulu, HI, USA, 27-30 Oct. 2002, pp. 966–970.

[15] I. Sutskover, S. Shamai, and J. Ziv, "Iterative decoding of low-density parity-check codes over compound channels," in *Proc. of IEEE 22nd Convention of Elec. Engineers in Israel*, Tel-Aviv, Israel, Dec. 2002, pp. 265–267.

[16] T. Friedlander and D. Burshtein, "Decoding algorithms for LDPC codes transmitted over channels with isi," in *Proc. of IEEE 22nd Convention of Elec. Engineers in Israel*, Tel-Aviv, Israel, Dec. 2002, p. 268.

[17] N. Wiberg, "Codes and decoding on general graphs," Ph.D. dissertation, Dept. of Electrical Engineering, Linköping, Sweden, 1996.

[18] R. Gallager, "Low-density parity-check codes," Cambridge, Massachusetts: M.I.T. Press, 1963.

[19] H. Jin and T. Richanrdson, "Design of low-density parity-check codes for noncoherent MPSK communication," in *Proc. of ISIT 2002*, 2002, p. 169.

[20] C. Wang and D. Sklar, "Performance of a turbo coded system with DPSK modulation using enhanced decoding metrics and matched channel side information," in *Proc. of the 2002 IEEE International Conference on Communications (ICC)*, 2002, pp. 773–777.

[21] B. Kurkoski, P. H. Siegel, and J. K. Wolf, "Joint message-passing decoding of LDPC codes and partial-response channels," *IEEE Trans. on Information Theory*, vol. 48, no. 6, pp. 1410–1422, June 2002.

[22] J. Garcia-Frais, "Decoding of low-density parity-check codes over finite-state binary markov channels," in *Proc. of ISIT 2001*, Washington DC, June 2001, p. 72.

[23] M. Luby, M. Mitzenmacher, M. Shokrollahi, and D. Spielman, "Analysis of low density codes and improved designs using irregular graphs," in *Proceedings of the 30th Annual ACM Symposium on the Theory of Computing*, 1998, pp. 249–258.

[24] T. Cover and J. Thomas, *Elements of Information Theory*. Wiley, 1991.

[25] H. Shin, S. Kim, and J. H. Lee, "Turbo decoding in a rayleigh fading channel with estimated channel state information," in *Proc. of VTC 2000*, 2000, pp. 1358–1363.

[26] S. ten Brink, "Code doping for triggering iterative decoding convergence," in *Proc. of International Symposium on Information Theory 2001, Washington, DC.*, June 2001, p. 235.

[27] T. Richardson, A. Shokrollahi, and R. Urbanke, "Design of capacity-approaching irregular low-density parity-check codes," *IEEE Trans. on Information Theory*, vol. 47, no. 2, pp. 619–637, Feb. 2001.
[28] M. Wang, W. Xiao, and T. Brown, "Soft decision metric generation for QAM with channel estimation error," *IEEE Trans. on Communications*, vol. 50, no. 7, pp. 1058–1061, July 2002.
[29] R. Krichevsky and V. Trofimov, "The performance of universal encoding," *IEEE Trans. on Information Theory*, vol. 27, no. 2, pp. 199–207, March 1981.
[30] T. Friedlander, "Decoding algorithms for LDPC codes transmitted over channels with isi," Master's thesis, Dept. of Electrical Engineering, Tel-Aviv University, Israel, 2002.
[31] K. Narayanan, X. Wang, and G. Yue, "LDPC code design for turbo equalization," in *Proc. of ITW 2002*, Bangalore, India, Oct. 2002, pp. 57–60.
[32] F. MacWilliams and N. Sloane, *The Theory of Error-Correcting Codes*. North-Holand, 1977.
[33] C.-C. Wang, S. Kulkarni, and H. Poor, "Density evolution for asymmetric memoryless channels," a preprint.

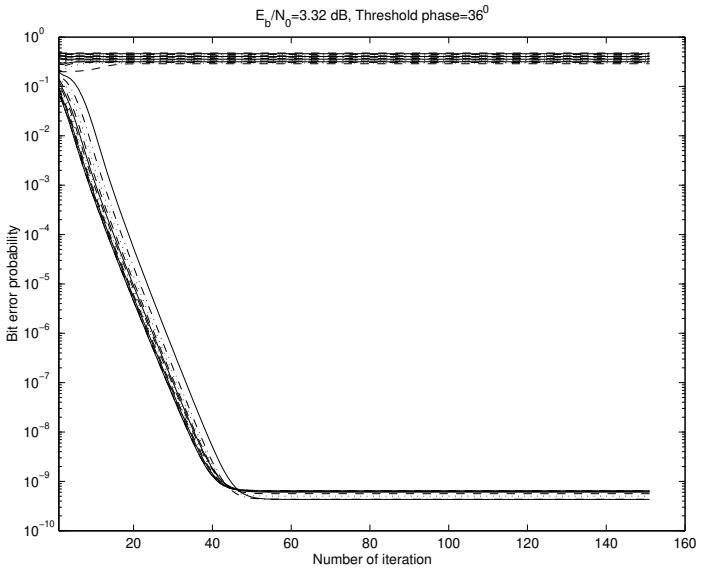


Fig. 5. Density evolution results for the response of Algorithm 2 to different phases, at high E_b/N_0.

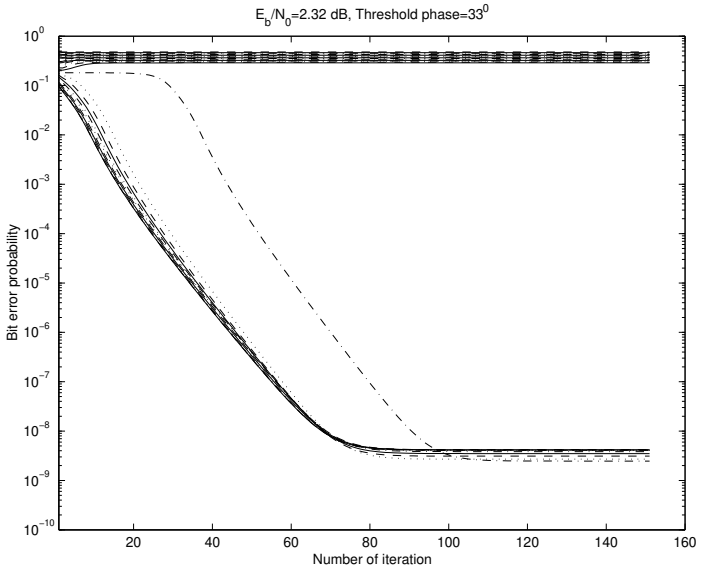


Fig. 6. Density evolution results for the response of Algorithm 2 to different phases, at medium E_b/N_0.

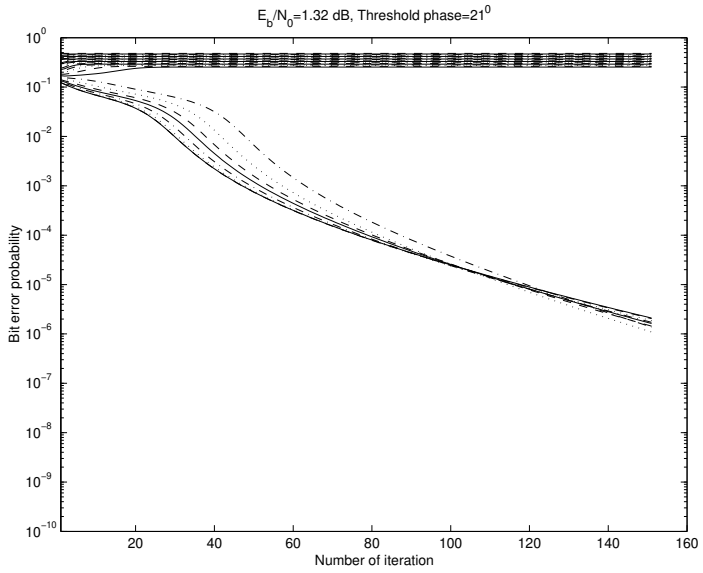


Fig. 7. Density evolution results for the response of Algorithm 2 to different phases, at low E_b/N_0.

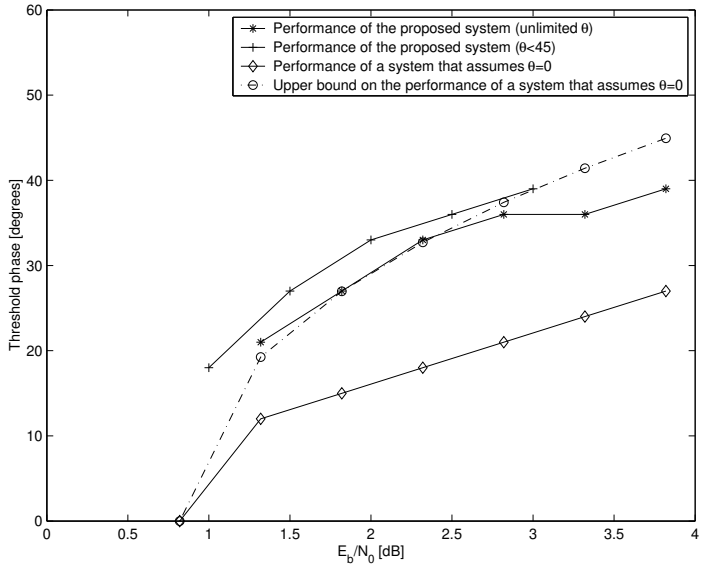


Fig. 8. Phase thresholds vs. E_b/N_0 for ensemble $\lambda_2 = 0.37, \lambda_4 = 0.63, \rho_5 = 0.5, \rho_7 = 0.5, \eta_{20} = 1.0$ and different types of grids.